

Scan & Pay - API Implementation

This document lists every API call needed to build the Scan & Pay flow, in order.

All our own APIs are marked OUR API.

Step 1: Customer scans QR - create or find session

OUR API POST /api/v1/sessions

Request:

```
{
  "qrId": "QR-POOL-T12",
  "hotelId": "REFUSCENT",
  "locationId": "POOL-BAR",
  "outletCode": "POOL01",
  "servicePointId": "T12"
}
```

Response:

```
{
  "sessionId": "S123",
  "status": "ACTIVE",
  "checkNumber": null
}
```

Step 2: Get menu

OUR API GET /api/v1/outlets/{outletCode}/menu

Calls: Touche GetOutletDetails GET /ToucheLite/GetOutletDetails Returns ItemCode, ItemName, Price.

NEEDS CONFIRMATION: full request parameters, auth format, whether item availability is included.

Our response to the app (already mapped to our own item IDs):

```
{
  "outletCode": "POOL01",
  "items": [
    { "itemId": "1001", "name": "Club Sandwich", "price": 45, "currency": "AED" },
    { "itemId": "1002", "name": "Iced Tea", "price": 20, "currency": "AED" }
  ]
}
```

Note: We keep a mapping table (itemId <-> Touche ItemCode) in our own database. This needs to be filled in per outlet.

Step 3: Place first order

OUR API POST /api/v1/orders

Request:

```
{
  "sessionId": "S123",
  "items": [
    { "itemId": "1001", "quantity": 2 },
    { "itemId": "1002", "quantity": 1 }
  ]
}
```

Internally: map itemId -> Touche ItemCode, then call:

Touche POST /ToucheLite/SaveOrder Request: TableNumber, ItemCode, Quantity, CheckNumber (empty on first order) Response: CheckNumber, Balance, CheckStatus (PO = open)

NEEDS CONFIRMATION: exact field names, auth headers, how outlet/table are passed.

Our response:

```
{
  "sessionId": "S123",
  "checkNumber": "M0015803",
  "checkStatus": "PO",
  "balance": 50,
  "currency": "AED"
}
```

Important: store sessionId -> checkNumber. This link is used for every step after this.

Step 4: View bill (optional, any time)

OUR API GET /api/v1/sessions/{sessionId}/bill

Calls: Touche GetAllOpenChecks (CONFIRMED name) GET /ToucheLite/GetAllOpenChecks

NEEDS CONFIRMATION: whether this can be filtered to one check, or returns all open checks for the outlet (in which case we filter it ourselves).

Our response:

```
{
```

```
"sessionId": "S123",  
"checkNumber": "M0015803",  
"checkStatus": "PO",  
"amount": 110,  
"currency": "AED"  
}
```

Step 5: Customer scans the QR again

OUR API POST /api/v1/sessions/resolve

Request:

```
{ "qrId": "QR-POOL-T12" }
```

Response:

```
{  
  "sessionId": "S123",  
  "servicePointId": "T12",  
  "checkNumber": "M0015803",  
  "checkStatus": "PO"  
}
```

Rule: this must never create a new session or a new check while checkStatus is PO. The app then shows two choices: "Add more items" or "View bill / Pay".

Step 6: Add more items (same check)

Same API as Step 3: OUR API POST /api/v1/orders

The only difference: our backend passes the existing checkNumber (M0015803) into SaveOrder, instead of an empty one.

Touche SaveOrder is called again, same API as Step 3, same checkNumber.

Response:

```
{  
  "sessionId": "S123",  
  "checkNumber": "M0015803",  
  "checkStatus": "PO",  
  "totalOrderPrice": 110,  
  "currency": "AED"
```

```
}
```

Rule: our backend decides new-check vs existing-check on its own, by checking if session.checkNumber is already set. The app never has to decide this.

Step 7: Start payment

OUR API POST /api/v1/payments

Request:

```
{
  "sessionId": "S123",
  "checkNumber": "M0015803",
  "amount": 110,
  "currency": "AED",
  "method": "AED"
}
```

Response:

```
{
  "paymentId": "PAY123",
  "status": "PENDING"
}
```

The "method" field decides which adapter to use next: "AED" goes to NI, "ZC" goes to ZC Wallet.

Step 8a: Payment via NI (AED)

These are internal calls our backend makes. The app never sees them directly, it only checks payment status.

1. GET_STATUS
2. GET_TRANSACTION_TYPES - must be called to find out which transaction types are actually enabled
3. START_TRANSACTION
4. GET_TRANSACTION_RESULT - used if the result doesn't come back immediately

NEEDS CONFIRMATION for all four: full request/response schema, field names, terminal/device binding.

Internal result our backend produces:

```
{
  "paymentId": "PAY123",
```

```
"provider": "NI",  
"transactionId": "NI-TXN-99881",  
"amount": 110,  
"currency": "AED",  
"status": "SUCCESS"  
}
```

Step 8b: Payment via ZC Wallet

Everything here is a placeholder. Do not build against these names, they only describe the shape needed.

Required from ZC before this can be built:

- Authentication
- Create payment/request API
- Payment status API
- Callback/webhook
- Refund API
- Cancel/expiry API
- Wallet/payment reference format

Build a generic adapter interface now so the rest of the system does not wait on ZC docs:

```
interface PaymentAdapter {  
    startPayment(paymentId, amount, currency) -> providerRef  
    getStatus(providerRef) -> SUCCESS | FAILED | PENDING  
    refund(providerRef, amount) -> refundRef  
}
```

NI and ZC both implement this interface later.

Step 9: Handle payment result

On SUCCESS:

- Set Payment.status = SUCCESS, store transactionId
- Go to Step 10 (settle POS check)

On FAILURE:

- Set Payment.status = FAILED
- Do not call SettleOrder
- Leave checkStatus as PO (open)

- Allow retry:

OUR API POST /api/v1/payments/{paymentId}/retry Creates a new payment attempt linked to the same checkNumber. Does not touch the POS check.

Duplicate payment protection:

- Every POST /api/v1/payments call should take a client-generated idempotencyKey. If the same key is sent again, return the existing paymentId instead of creating a new payment.
- Before calling SettleOrder, check that Payment.status is SUCCESS and Check.status is not already ST. This prevents double settlement if something fires twice.
- Log every NI/ZC call (request, response, timestamp) in a PaymentTransaction table for reconciliation.

Step 10: Settle the POS check

Touche SettleOrder (CONFIRMED name) POST /ToucheAPI/ToucheLite/SettleOrder

Called only after payment is SUCCESS.

Request:

```
{
  "checkNumber": "M0015803",
  "outlet": "POOL01",
  "amount": 110,
  "settlementOptionCode": "??",
}
```

NEEDS CONFIRMATION: the correct settlementOptionCode for NI/3C payments. Do not assume CASH or any existing code works, this must be confirmed against Touche config for this property.

NEEDS CONFIRMATION: full request/response schema beyond what's listed above.

Expected result: CheckStatus moves from PO to ST (settled/closed).

Our backend then updates:

```
{
  "checkNumber": "M0015803",
  "checkStatus": "ST",
  "sessionStatus": "CLOSED",
  "paymentId": "PAY123"
}
```